Linear & Polynomial

Machine Learning Workflow: Linear & Polynomial Regression

This guide covers the end-to-end process of building a regression model, from data cleaning to visualization.

1. Import Essential Libraries

The foundation of any data science project involves data manipulation, numerical operations, and plotting.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn import metrics
```

2. Data Loading & Cleaning

Before modeling, we must ensure the data is clean and in numerical format.

```
# Load the dataset
df = pd.read_csv('file_name.csv')

# Check for missing values
df.isna().sum()

# Convert Categorical (text) data to Numerical
# drop_first=True prevents the "Dummy Variable Trap"
df = pd.get_dummies(df, drop_first=True)

# Remove unnecessary columns
df.drop(['column_name'], axis=1, inplace=True)
```

3. Feature Engineering (Polynomial Regression)

If the relationship between data points is not a straight line, we use **Polynomial Features** to create a curve.

```
# x = Independent variables, y = Target variable
x = df[['feature']]
y = df['target']

# Transform x into polynomial terms (degree 2 = quadratic, 3
= cubic)
poly = PolynomialFeatures(degree=2)
x_poly = poly.fit_transform(x)
```

4. Data Splitting

We split data into a **Training set** (to teach the model) and a **Testing set** (to verify its accuracy).

```
# train_size=0.8 means 80% for training and 20% for testing
x_train, x_test, y_train, y_test = train_test_split(x_poly,
y, train_size=0.8, random_state=1)

# Verify the shape
print(f"Train shape: {x_train.shape}, Test shape: {x_test.sha
pe}")
```

5. Model Building & Training

This is where the computer "learns" the patterns from the training data.

```
# Initialize and train the model
lr = LinearRegression()
lr.fit(x_train, y_train)

# Predict values using the test set
ypred = lr.predict(x_test)
```

6. Model Evaluation

After predicting, we compare the results to actual values and check the math behind the model.

```
# Create a Comparison Table
diff = pd.DataFrame({"Actual": y_test, "Predicted": ypred})
print(diff.head())

# Check Model Parameters
print(f"Coefficients: {lr.coef_}")
print(f"Intercept: {lr.intercept_}")

# R2 Score (Accuracy): 1.0 is a perfect fit
```

```
score = metrics.r2_score(y_test, ypred)
print(f"R2 Score: {score}")
```

7. Visualization

Visualizing the results helps communicate how well the model "fits" the data.

For Linear Regression (Straight Line)

```
plt.scatter(x, y, color='red', label='Actual Data') # Plot da
ta points
plt.plot(x, lr.predict(poly.fit_transform(x)), color='blue',
label='Regression Line') # Plot the line
plt.title('Actual vs Predicted')
plt.xlabel('Feature Name')
plt.ylabel('Target Name')
plt.legend()
plt.show()
```

Key Concepts to Remember for your Presentation:

1. `isna().sum()`: Always check for "NaN" (Not a Number) values first, or the model will crash.
2. `get_dummies`: Models cannot read strings (like "Male/Female"). We must convert them to numbers (0 and 1).
3. `PolynomialFeatures`: Use this when a simple straight line is too "stiff" to capture the data's curve.
4. `r2_score`: This tells you what percentage of the variance in the target is explained by your model. Higher is better!
5. **Scatter vs Plot**:
   - `plt.scatter` is used for the **dots** (the real-world data).
   - `plt.plot` is used for the **line/curve** (the model's prediction).